

A Multi-Gb/s Frame-Interleaved LDPC Decoder With Path-Unrolled Message Passing in 28-nm CMOS

Mario Milicevic^{ID}, *Member, IEEE*, and P. Glenn Gulak, *Senior Member, IEEE*

Abstract—This paper presents a frame-interleaved low-density parity-check (LDPC) decoder architecture with a new interconnect partitioning scheme and time-distributed Min-Sum decoding schedule. The architecture exploits the cyclic structure of the parity-check matrix by unrolling each check node update in time over all connected variable nodes in order to minimize wiring complexity and power. Multiple frames are interleaved to maximize hardware utilization, while coarse-grained clock gating is used to systematically turn off inactive logic and memories to save power. To demonstrate the scalability of the proposed architecture, a multirate LDPC decoder test chip was fabricated for the IEEE 802.11ad standard in the 28-nm CMOS technology node. The design occupies an area of 1.99 mm², contains 160 Kb of embedded static random-access memory, and achieves a throughput of 6.78 Gb/s at 10 decoding iterations for all four code rates specified in the standard. With early decoding termination, the fabricated chip consumes between 104 and 279 mW of power at a target bit error rate of 10^{−6} under nominal operation at 0.9-V supply and 202-MHz clock rate, resulting in an energy efficiency between 1.53 and 4.12 pJ/bit/iteration. With clock-frequency and voltage scaling, the fabricated chip achieves an energy efficiency between 1.1 and 3.1 pJ/bit/iteration. This paper achieves the highest normalized energy efficiency among recently published CMOS-based decoders for the IEEE 802.11ad standard at nominal clock frequency and supply voltage.

Index Terms—IEEE 802.11ad, low-complexity interconnect, low-density parity-check (LDPC), LDPC decoder, Min-Sum, multi-Gb/s, pipeline frame interleaved, quasi-cyclic (QC).

I. INTRODUCTION

LOW-DENSITY parity-check (LDPC) codes have been widely adopted over the past 15 years for forward error correction (FEC) in wireless, wireline, optical, and non-volatile memory systems due to their near-Shannon limit error-correction performance and the absence of patent licensing fees [1]–[4]. Their adoption in modern standards, such as

IEEE 802.11ac (Wi-Fi), was predominantly enabled through the introduction of hardware-friendly variants of the belief propagation decoding algorithm [5], hardware-oriented codes, and integrated circuit decoder architectures capable of delivering multi-Gb/s throughput at low power. Today, the two key design constraints of LDPC decoders are: energy efficiency and system-on-chip (SoC) integration capability. The scalability of LDPC decoders is primarily limited by the complexity of message-passing interconnect between on-chip memory and parallel processing units [6], [7]. Due to the recent stagnation of wired interconnect scaling beyond the 45-nm CMOS technology node, unstructured routing now largely dominates overall power consumption [8], [9].

A practical design constraint of an LDPC decoder is the SoC integration capability of the intellectual property (IP) core alongside other signal processing blocks in a digital baseband implemented in a modern bulk CMOS technology. While clock frequencies in excess of 500 MHz are achievable in digital blocks in sub 28-nm CMOS technology nodes, the digital baseband in most production-level, industry-grade, communications/networking SoCs typically operates at clock frequencies in the 200–300-MHz range to meet timing constraints across process, voltage, and temperature (PVT) corners [10]. As such, the clock frequency of the LDPC decoder should realistically be constrained to be around 200 MHz. Under this condition, achieving multi-Gb/s throughput is a challenge using traditional architectural and scheduling techniques [11], [12], especially for short block-length codes such as those defined in the IEEE 802.11ad (WiGig) standard [13]. While multiple low-throughput LDPC decoders can be instantiated in parallel to achieve multi-Gb/s throughput, their aggregate power may exceed the FEC power budget in an industry-grade SoC.

The motivation of this paper is to address the high power consumption of unstructured interconnect in multi-Gb/s decoders by exploiting the structure of LDPC parity-check matrices to reduce wiring complexity through new architectural techniques that scale to sub-10-nm CMOS technology nodes. This paper investigates a new multi-Gb/s LDPC decoder architecture, which leverages the low cost of transistor area for a single IP block relative to the overall SoC, when implemented in a modern CMOS technology node. The proposed architecture is scalable by design and is based on dark silicon design principles [14], where a high transistor area offers high computational parallelism at a low clock rate to maximize throughput (when required), while systematic clock gating turns off inactive logic to minimize switching power.

Manuscript received December 3, 2017; revised March 28, 2018; accepted May 7, 2018. Date of publication June 14, 2018; date of current version September 25, 2018. This work was supported in part by MaxLinear Inc., CA, USA, in part by the Natural Science and Engineering Research Council of Canada, and in part by the Canadian Microelectronics Corporation. (Corresponding author: Mario Milicevic.)

M. Milicevic was with the Department of Electrical and Computer Engineering, University of Toronto, Toronto, ON M5S 3G4, Canada. He is now with MaxLinear Inc., Irvine, CA 92618 USA (e-mail: mario.milicevic@utoronto.ca).

P. G. Gulak is with the Department of Electrical and Computer Engineering, University of Toronto, Toronto, ON M5S 3G4, Canada (e-mail: gulak@eecg.toronto.edu).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TVLSI.2018.2838591

1063-8210 © 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

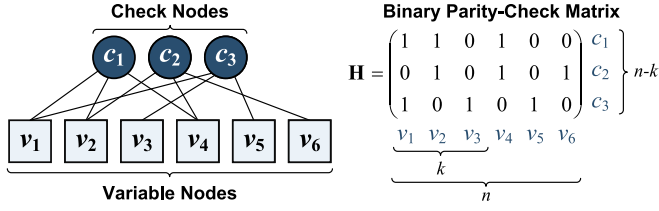


Fig. 1. Tanner graph and corresponding parity-check matrix $\mathbf{H}$ with block length of $n = 6$ bits, $k = 3$ information bits, $n = 6$ VNs, and $(n-k) = 3$ CNs. CNs and VNs correspond to the rows and columns of $\mathbf{H}$, respectively.

This paper presents a new, frame-interleaved LDPC decoder architecture, where long interconnect wires are split into multiple shorter segments through a reformulated message-passing schedule to reduce interconnect delay and routing logic overhead, by exploiting the spatial locality of stored messages in neighboring processing nodes. The architecture supports multiple code rates and achieves multi-Gb/s throughput while operating at clock rates below 200 MHz. The IEEE 802.11ad standard for the 60-GHz millimeter-wave band is used as a vehicle to demonstrate the application of the proposed architecture, due to its multi-Gb/s throughput specification over four code rates, and quasi-cyclic (QC) LDPC parity-check matrix structure [13]. A proof-of-concept application-specific integrated circuit (ASIC) was fabricated in a 28-nm CMOS technology and tested at-speed on a commercial SoC tester. The LDPC decoder achieves a throughput of 6.78 Gb/s with a normalized energy efficiency of 1.53 pJ/bit/iteration for the rate 13/16 code, and 4.12 pJ/bit/iteration for the rate 1/2 code, while operating at a 0.9-V supply and 202-MHz clock rate.

The remainder of this paper is organized as follows. Section II provides a background on LDPC decoding and outlines challenges in traditional decoder architectures. Section III introduces the frame-interleaved architecture with a path-unrolled message-passing schedule. Section IV presents the silicon chip results and comparison to other published works.

II. BACKGROUND

LDPC codes are a class of linear block codes defined by a sparse parity-check matrix $\mathbf{H}$ of size $(n-k) \times n$, with block length n and code rate $R_{\text{code}} = k/n$ [2], [15]. As shown in Fig. 1, an LDPC code is equivalently defined by its Tanner graph, where the nonzero entries of the binary parity-check matrix $\mathbf{H}$ define the edge connections between independent vertex sets known as check nodes (CNs) and variable nodes (VNs) [16]. LDPC decoding produces an estimate $\hat{\mathbf{C}}$ of an LDPC-encoded codeword $\mathbf{C}$ that was transmitted over a noisy channel, and is considered successful if $\hat{\mathbf{C}} = \mathbf{C}$.

A. LDPC Decoding: Belief Propagation

LDPC decoding is performed via belief propagation, an iterative message-passing algorithm, which attempts to converge on a valid codeword by iteratively exchanging updates between CNs and VNs until a valid codeword is found, or the maximum number of iterations is exhausted [17]. The Min-Sum algorithm has been adopted as a suitable approximation to the traditional Sum-Product algorithm, as it can be computed with simple comparator circuits [6], [15].

Algorithm 1 Min-Sum Decoding With Flooding Schedule

Input: $\mathbf{y}, \sigma^2$; **Output:** $\hat{\mathbf{C}}$

Step 1: LLR initialization at each VN, $v = 1, 2, \dots, n$

$$Q_v \leftarrow \ln \left(\frac{P(y_v | C_v = 0)}{P(y_v | C_v = 1)} \right) = \frac{2y_v}{\sigma^2} \text{ for BIAWGNC}$$

$L_{vc}^{(i=0)} \leftarrow Q_v, \forall c \in M(v)$ for first iteration

for Iteration $i = 1$ to Max Iterations **do**

Step 2: Check node update at each CN, $c = 1, 2, \dots, n-k$

$$m_{cv}^{(i)} \leftarrow \left(\prod_{v' \in N(c) \setminus v} \text{sgn}(L_{v'c}^{(i-1)}) \right) \times \min |L_{v'c}^{(i-1)}|$$

Step 3: Variable node update at each VN v

$$L_v^{(i)} \leftarrow Q_v + \sum_{c \in M(v)} m_{cv}^{(i)}$$

$$L_{vc}^{(i)} \leftarrow Q_v + \sum_{c' \in M(v) \setminus c} m_{c'v}^{(i)} = L_v^{(i)} - m_{cv}^{(i)}$$

Step 4: Hard decision at each VN v

$$\hat{C}_v^{(i)} \leftarrow \begin{cases} 0, & L_v^{(i)} \geq 0 \\ 1, & \text{otherwise} \end{cases}$$

Step 5: Early termination (parity) check at each CN

if $\hat{\mathbf{C}}\mathbf{H}^T = \mathbf{0} \pmod{2}$ **then** Terminate

end for

Algorithm 1 describes the Min-Sum decoding procedure with a flooding schedule, where CNs and VNs pass message updates between their connected neighbors once per iteration to generate a codeword estimate $\hat{\mathbf{C}}$. The channel is assumed to be a binary-input additive white Gaussian noise channel (BIAWGNC) with zero mean and noise variance σ^2 , $\mathbf{y}$ is the received soft-decision vector of length n , Q_v is the log-likelihood ratio (LLR) input value at VN v , m_{cv} is the message from CN c to VN v , and L_{vc} is the message from VN v to CN c . The set of VNs connected to CN c is defined as $N(c) = \{v | v \in \{1, 2, \dots, n\} \wedge H_{cv} = 1\}$, where $v' \in N(c) \setminus v$ denotes all VNs in $N(c)$ excluding VN v . The set of CNs connected to VN v is defined as $M(v) = \{c | c \in \{1, 2, \dots, n-k\} \wedge H_{cv} = 1\}$, where $c' \in M(v) \setminus c$ refers to all CNs in $M(v)$ excluding CN c . The syndrome $\hat{\mathbf{C}}\mathbf{H}^T = \mathbf{0}$ if the parity check $p_c = 0$ at each CN c , where p_c is the XOR of all hard decision $\hat{C}_v$ bits from VNs connected to CN c in the set $N(c)$:

$$p_c \leftarrow \hat{C}_{N(c)_1} \oplus \hat{C}_{N(c)_2} \oplus \dots \oplus \hat{C}_{N(c)_{|N(c)|}} = \bigoplus_{v \in N(c)} \hat{C}_v. \quad (1)$$

The Normalized and Offset Min-Sum algorithms offer improvements in the bit error rate (BER) performance over the traditional Min-Sum algorithm by introducing scaling and offset factors, respectively [18]. However, these parameters are strongly dependent on the channel signal-to-noise ratio (SNR), code rate, and fixed-point message quantization. To avoid BER degradation, these parameters should be adjusted dynamically during runtime to compensate for changes in SNR or code rate. Such enhancements are beyond the scope of this paper.

B. Quasi-Cyclic LDPC Codes

Quasi-Cyclic LDPC codes are a class of architecture-aware codes, where the parity-check matrix is constructed from an array of $q \times q$ cyclically-shifted identity matrices or $q \times q$ zero matrices to reduce decoder implementation complexity [19], [20]. The tilings evenly divide the $\mathbf{H}$ matrix into n/q QC macrocolumns and $(n-k)/q$ QC macrorows. QC matrices have a highly regular structure, which can be exploited to simplify decoder architecture design.

C. LDPC Decoder Architectures

LDPC decoder architectures can be classified into the following three categories based on their degree of computational parallelism: fully parallel, partially parallel, or serial. Fully-parallel decoders achieve the highest throughput, but suffer from high power consumption and a large silicon area due to the large number of parallel processors, complex reconfigurable networks, and wired interconnect [21]–[23]. Partially-parallel decoder architectures achieve higher energy efficiency and area efficiency at the expense of lower throughput, as message updates are computed in a time-multiplexed approach, and barrel shifters are typically used to perform cyclic data rotation [19], [24]. Serial architectures implement a single processing unit with large memory and are suitable only for low-throughput applications, where high latency is acceptable [25].

D. Flooding and Layered Decoding Schedules

LDPC decoders execute the belief propagation algorithm based on either a flooding or layered message-passing schedule [23]. In the flooding schedule, CNs and VNs perform subsequent rounds of updates and pass messages between their connected neighbors once per iteration [23]. Flooding decoders achieve high throughput with low latency at the expense of area and routing complexity [26]. The layered schedule improves decoder convergence by performing intermediate LLR updates in each iteration [27]. While the layered schedule generally requires fewer iterations, the overall decoding latency may be longer as additional clock cycles are required to perform intermediate LLR updates. These successive message updates can also lead to dynamic range clipping/saturation, which reduces error-correction performance if scaling circuits are not employed. Moreover, the data dependence among layers causes memory access contention [11], [28], [29]. Layered decoders require higher clock rates to achieve multi-Gb/s throughput and more complex permutation network control. As such, the layered schedule is not optimal for a multi-Gb/s decoder with a target clock rate of 200 MHz.

E. Energy-Efficiency Challenges in CMOS-Based Decoders

Reducing interconnect congestion and minimizing reconfigurable routing logic are key challenges in improving decoder energy efficiency. The split-row technique reduces the silicon area and routing congestion in fully parallel decoders [6], but is suitable only for single-rate codes with high row weights. The half-row decoding schedule folds the traditional layered schedule computations in time to reduce interconnect wiring [30], but suffers in throughput and introduces more complex data shuffling circuits. Memory-based decoders enable frame-level pipelining with shorter routing wires [31], while row-merging techniques can be applied to improve decoding throughput with reconfigurable routing logic overhead [32].

Refresh-free dynamic memory implementations have been proposed to reduce memory power. Transistor-based embedded dynamic random-access memory (eDRAM) can be used without special process requirements [33]; however, the higher supply voltage of eDRAM increases SoC design complexity. Dynamic standard-cell memory can reduce area overhead [34];

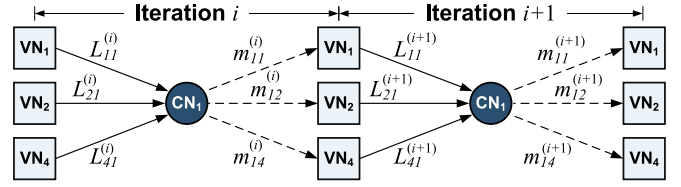


Fig. 2. Explicitly defined processing units VN_1 , VN_2 , and VN_4 connected to processing unit CN_1 based on the Tanner graph in Fig. 1 with L_{vc} and m_{cv} messages indicated for decoding iterations i and $i+1$.

however, the cell retention time is susceptible to PVT variations, particularly in newer technology nodes where leakage is higher.

Time-domain signal processing techniques have been explored to shorten the critical path in CN and VN updates [35], [36]. However, such implementations also demonstrate timing closure challenges due to metastability, and PVT variations in the delay chains and custom clock-tree networks.

F. Explicit Check and Variable Node Processing Units

The two-phase decoding schedule allows check and variable updates to be mapped to explicit CN and VN processing units. Fig. 2 presents a dataflow diagram of L_{vc} and m_{cv} messages between CN_1 and its connected VNs. In each iteration, CN_1 receives a unique L_{vc} message from each VN, and then calculates a unique m_{cv} return message for each VN. This canonical mapping is adopted in most decoder implementations; however, it requires all L_{vc} and m_{cv} messages to be routed between the two processing groups through a congested interconnect network, with routing permutation logic in both the CNs and VNs. The complexity of the interconnect network and permutation logic scales with block length. The m_{cv} computation in Step 2 of Algorithm 1 can be calculated using only the first and second minimum magnitudes as follows:

$$m_{cv} \leftarrow \begin{cases} (s_c \times \text{sgn}(L_{vc})) \times \min 2_c, & \text{if } \min 1_c = |L_{vc}| \\ (s_c \times \text{sgn}(L_{vc})) \times \min 1_c, & \text{otherwise} \end{cases} \quad (2)$$

$$s_c \leftarrow \prod_{v \in N(c)} \text{sgn}(L_{vc}) \quad (3)$$

$$\min 1_c \leftarrow \min_{v \in N(c)} |L_{vc}| \quad (4)$$

$$\min 2_c \leftarrow \min_{v' \in N(c) \setminus \{v_{\min 1}\}} |L_{v'c}|. \quad (5)$$

However, high-order compare-select trees are still required. Pipeline registers can be added to relax the critical path timing constraints in the compare-select trees and CN-to-VN interconnect; however, the scalability of the explicitly partitioned architecture remains limited. New architectural and scheduling techniques are therefore required to alleviate the global routing and energy efficiency challenges.

III. PROPOSED LDPC DECODER ARCHITECTURE

The proposed LDPC decoder architecture partitions the global message-passing network into structured, local interconnect groups defined between successive macrocolumns of the QC parity-check matrix. A time-distributed decoding schedule is introduced to exploit the spatial locality of update messages by combining the CN- and VN-phase update logic into a single processing unit with deterministic memory access.

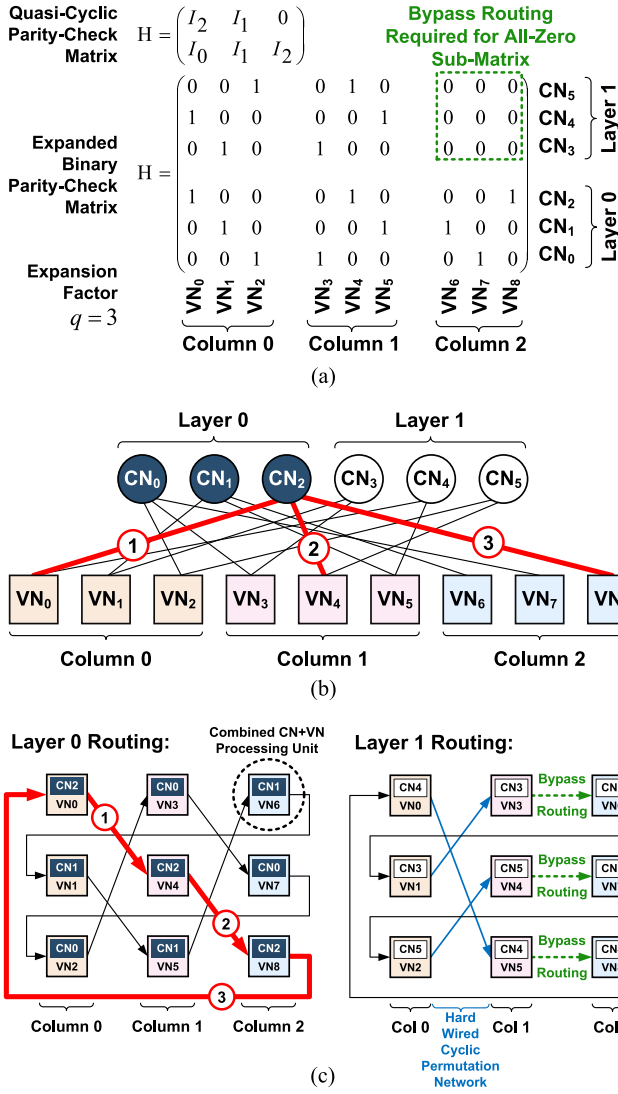


Fig. 3. Simplified example of the proposed LDPC decoder architecture based on (a) sample two-layer QC parity-check matrix, and (b) Tanner graph with one decoding path highlighted. The arrangement of combined CN+VN processing units in the systolic array architecture and routing patterns in Layer 0 and Layer 1 are shown in (c). Messages in each layer are transmitted sequentially between connected CN+VN units in successive columns. In each column, all the layers inside each CN+VN unit are processed in parallel. In (c), the routing in Layer 0 is defined by three closed paths that correspond to the VN connectivity of CN_0 , CN_1 , and CN_2 . In Layer 1, the routing is defined by the VN connectivity of CN_3 , CN_4 , and CN_5 . Thus, each CN has a unique routing pattern. The highlighted closed path given by $VN_0-CN_2-VN_4-CN_2-VN_8-CN_2-VN_0$ in (b) is unrolled in (c) such that CN_2 is absorbed into its connected VNs, resulting in the following unrolled path: $VN_0-VN_4-VN_8-VN_0$.

The reformulation of the Min-Sum flooding schedule leverages the QC interconnect structure to minimize routing congestion and permutation logic overhead, while enabling frame-level parallelism for multirate, multi-Gb/s decoding.

A. Hardware Mapping With Path-Unrolled Decoding Schedule

The proposed decoding schedule unrolls the CN update in time by distributing the CN-to-VN m_{cv} message computation across all of the VNs that participate in the CN update. This

differs from the traditional approach, where all m_{cv} updates for CN c are computed in a single CN processing unit. Fig. 3 presents an illustrative example of the interconnect routing patterns and combined processing unit arrangement for the proposed architecture. For the QC matrix defined in Fig. 3(a), the corresponding Tanner graph is shown in Fig. 3(b), where messages exchanged between connected CNs and VNs trace a closed path along the edges of the graph. As highlighted in Fig. 3(b), one such path is defined by the node sequence: $VN_0-CN_2-VN_4-CN_2-VN_8-CN_2-VN_0$. By unrolling this path, the intermediate CN_2 node can be removed by distributing the CN update operation among its connected VNs. This modification does not alter the result of the Min-Sum algorithm, but simply introduces a piecewise calculation of each CN-to-VN m_{cv} message defined in Step 2 of Algorithm 1.

Fig. 3(c) shows combined processing nodes arranged in column groups, which correspond to the macrocolumns of the QC matrix. The combined CN+VN processing units are indexed according to their VN ordering. Each combined CN+VN processor contains the original VN-update logic to compute Steps 3 and 4 of Algorithm 1, as well as intermediate CN-update logic for the unrolled CN-to-VN m_{cv} message computation. The interconnect between column groups is defined by the cyclic permutation between the connected VNs in adjacent columns. Each partitioned network between successive column groups is hard wired, such that all combined CN+VN processors along each unrolled path are connected to a single CN in the original Tanner graph. For example, the Tanner graph edges labeled in Fig. 3(b) trace the following unrolled path in Layer 0 in Fig. 3(c): $VN_0-VN_4-VN_8-VN_0$. Routing complexity is constrained between successive columns by serializing the large fan-in/fan-out to/from each CN.

The QC matrix guarantees that each VN is connected to at most one CN in each layer (macrorow) of the matrix. Hence, each layer of the QC matrix requires an independent routing layer in the proposed structure, as shown by Layers 0 and 1 routing patterns in Fig. 3(c). This is a realization of the flooding schedule and, thus, each combined CN+VN processor requires independent CN-update logic for each active processing layer. For example, in Fig. 3(c), the node corresponding to VN_0 contains CN-update logic for both CN_2 and CN_4 paths in Layers 0 and 1, respectively.

Bypass routing is required between successive columns in layers where an all-zero submatrix appears in the QC matrix, in order to maintain the continuity of the closed path in the message-passing interconnect and to guarantee an equal number of column hops in the path traversal. In the bypass mode, the processing node in the successive column is simply included in the path, and the CN- and VN-update computations are not performed. Bypass routing introduces an artificial edge into the Tanner graph, such that the number of CN+VN units in each closed path is equal. Section III-B describes the mathematical modification to the flooding Min-Sum algorithm as a result of the path-unrolled message-passing schedule.

B. Time-Distributed Piecewise Min-Sum Computation

The proposed architecture is a systolic array with homogeneous CN+VN processors. Each closed routing path is defined by the VNs connected to a single CN c , i.e., the VNs in the set $N(c)$. By absorbing the CN update among all connected

TABLE I

PIECEWISE TIME-DISTRIBUTED REFORMULATION OF MIN-SUM ALGORITHM WITH FLOODING SCHEDULE FOR SINGLE-LAYER ROUTING PATH

Iteration Phase	Column t	Parity $p_c(t)$	Sign $s_c(t)$	First Minimum Magnitude $\min1_c(t)$	Second Minimum Magnitude $\min2_c(t)$
Check Update Compute: $s_c(t)$ $\min1_c(t)$ $\min2_c(t)$	0	—	$\text{sgn}(L_{vc}(0))$	$ L_{vc}(0) $	[MAX LLR MAGNITUDE]
	1	—	$s_c(0) \times \text{sgn}(L_{vc}(1))$	$\min(L_{vc}(1) , \min1_c(0))$	$\min(L_{vc}(1) , \min1_c(0), \min2_c(0)) \setminus \{\min1_c(0)\}$
	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
	t	—	$s_c(t-1) \times \text{sgn}(L_{vc}(t))$	$\min(L_{vc}(t) , \min1_c(t-1))$	$\min(L_{vc}(t) , \min1_c(t-1), \min2_c(t-1)) \setminus \{\min1_c(t-1)\}$
	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
	$T-1$	—	$s_c(T-2) \times \text{sgn}(L_{vc}(T-1))$	$\min(L_{vc}(T-1) , \min1_c(T-2))$	$\min(L_{vc}(T-1) , \min1_c(T-2), \min2_c(T-2)) \setminus \{\min1_c(T-2)\}$
Variable Update Compute: $p_c(t)$ Propagate: $s_c(t)$ $\min1_c(t)$ $\min2_c(t)$	0	$\hat{C}_v(0)$	$s_c(T-1)$	$\min1_c(T-1)$	$\min2_c(T-1)$
	1	$p_c(0) \oplus \hat{C}_v(1)$	$s_c(T-1)$	$\min1_c(T-1)$	$\min2_c(T-1)$
	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
	t	$p_c(t-1) \oplus \hat{C}_v(t)$	$s_c(T-1)$	$\min1_c(T-1)$	$\min2_c(T-1)$
	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
	$T-1$	$p_c(T-2) \oplus \hat{C}_v(T-1)$	$s_c(T-1)$	$\min1_c(T-1)$	$\min2_c(T-1)$

VNs, the traditional CN-to-VN m_{cv} and VN-to-CN L_{vc} messages defined in Algorithm 1 are not explicitly computed, nor routed through the proposed structure. Instead, the L_{vc} and m_{cv} computations are discretized through a reformulation of the flooding Min-Sum algorithm. One decoding iteration requires two passes through the architecture. The first pass corresponds to the CN-update phase, and the second pass corresponds to the VN-update phase. Table I presents one decoding iteration of the piecewise, time-distributed Min-Sum schedule for a complete CN routing path over $T = n/q$ total columns.

In the first phase (CN update), the sign $s_c(t)$, first minimum magnitude $\min1_c(t)$, and second minimum magnitude $\min2_c(t)$ for every CN c are updated sequentially at every column t over T processing-node hops in T successive columns. Each combined processing unit stores its own L_{vc} value from the previous iteration. In each iteration, when the path traversal hop arrives at a particular node in column t , the internally stored L_{vc} value is used to update the intermediate sign and minimum magnitudes, which are then sent to the next successive column $t+1$. The path traversal then hops to the next connected processing node in the successive column, and the updates continue until the last column $T-1$ of the structure is reached, at which point, the final $s_c(T-1)$, $\min1_c(T-1)$, and $\min2_c(T-1)$ for CN c are known.

In the second phase (VN update), the sign and minimum magnitude values that were computed in the first CN-update pass are not updated any further, but rather held constant and broadcasted through the CN path to all connected processing units over T successive hops. Each processing unit first computes its own CN-to-VN m_{cv} message based on (2). The computed m_{cv} value is then used to calculate the intermediate LLR L_v , hard decision $\hat{C}_v$, and new VN-to-CN L_{vc} message based on Steps 3 and 4 of Algorithm 1. The updated L_{vc} value is stored in the processing unit's memory to be used in the next iteration. A piecewise parity computation is performed to eliminate the explicit parity check defined by Step 5 in Algorithm 1. The parity $p_c(t)$ for CN c is updated sequentially along the closed path, such that the final parity across all VNs

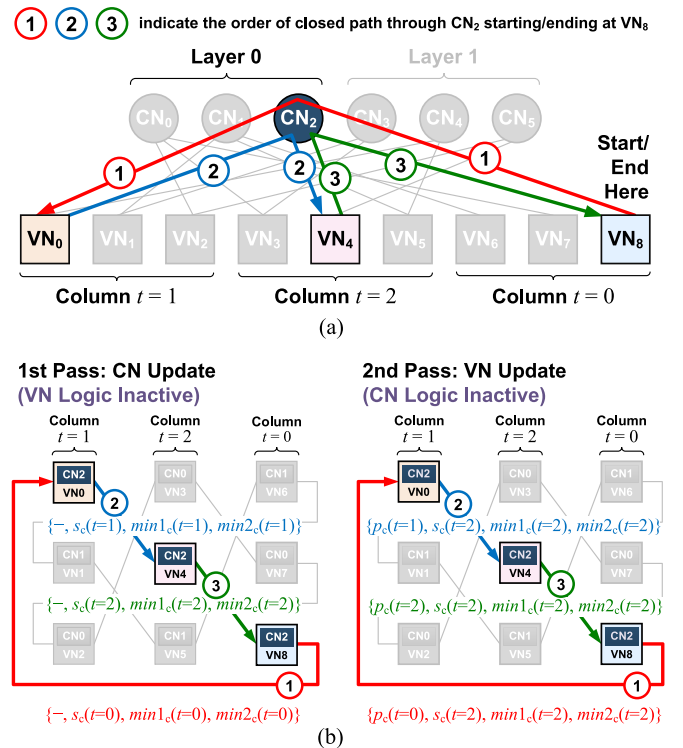


Fig. 4. (a) Closed path through CN₂ in the Tanner graph for one pass (phase) of decoding. (b) Unrolled piecewise messages that are passed between combined CN+VN processing units through Layer 0 routing in successive columns corresponding to the closed path highlighted in (a). Starting column position does not affect the final result of the piecewise distributed Min-Sum computation. Here, $t=0$ arbitrarily corresponds to the third column of $T=3$ total columns. The same decoding result would be obtained if $t=0$ was chosen to be the first or second column.

connected to CN c is determined by the last column of the path traversal. The parity result from the current iteration is known at the start of the CN-update phase of the next iteration.

Fig. 4 presents an example of the reformulated decoding procedure for one decoding iteration. A single layer message for CN₂ of the form $\{p_c(t), s_c(t), \min1_c(t), \min2_c(t)\}$

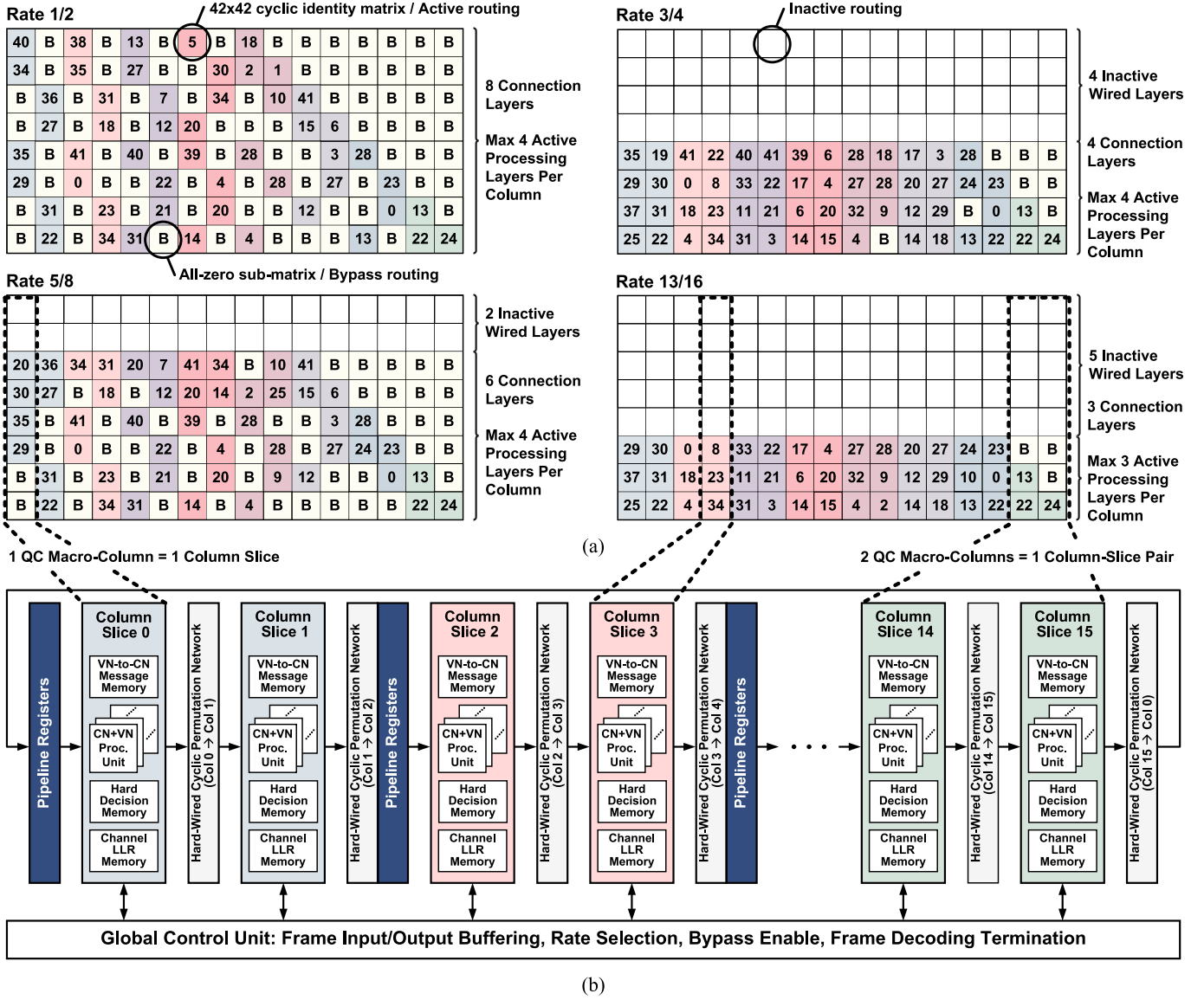


Fig. 5. (a) Columnwise partitioning of the IEEE 802.11ad QC parity-check matrices and their hardware mapping to the proposed architecture [13]. Each submatrix value indicates the cyclic permutation index of a 42×42 identity matrix. The four matrices are derived from a single 8-layer base matrix by removing layers in higher-rate matrices or by removing cyclically-shifted submatrices in lower rate matrices. (b) System block diagram for the proposed architecture showing the global control unit, and the datapath containing: column slices with combined CN+VN processing units and memories, a hard-wired cyclic permutation network between each column slice, and pipeline registers between column-slice pairs.

is successively consumed and updated in each column t of T total columns among the T combined CN+VN processing units along the closed CN_2 path. The final value of each of the four components at column $T - 1$ is given by (1) and (3)–(5). VN-update logic is inactive during the CN-update pass, and vice versa. In addition, column $t = 0$ does not necessarily correspond to the absolute first column of the QC matrix, but rather refers to the starting column for a particular layer message. Since the distributed updates in each column are independent, multiple frames can be interleaved in the structure to ensure a constant workload in order to maximize hardware utilization and avoid idle logic.

C. Parity-Check Matrix Partitioning and Hardware Mapping

The hardware mapping of the proposed architecture targets the peak throughput modes of the IEEE 802.11ad standard,

which specifies a fixed block length of 672 bits for four code rates, with a maximum throughput requirement of 6.757 Gb/s and maximum frame latency of $1 \mu s$ [13].

Fig. 5(a) presents the columnwise partitioning of the four QC matrices from the IEEE 802.11ad standard. As shown in Fig. 5(a), the QC matrix for each of the four code rates is derived from a single 8-layer base matrix. Each of the 16 macrocolumns maps directly to a column slice in the proposed architecture shown in Fig. 5(b), and maintains the same VN ordering such that each column slice contains 42 combined CN+VN processing units. Adjacent columns are connected through hard-wired routing in each of the 8 defined connection layers, which correspond to $8 \times 42 = 336$ CN paths.

The connectivity between successive columns is specific to the parity-check matrix. The interconnect mapping between

TABLE II
COLUMN-SLICE MESSAGES HIGHLIGHTED IN FIG. 6

Label	Message Description and Format
A, B	Input channel LLRs for VNs in current column slice t 5-bit message: $\{Q_v\} \times 42$ VNs
C	Incoming connection layer messages from column slice $t-1$ 10-bit msg: $\{p_c(t-1), s_c(t-1), \min1_c(t-1), \min2_c(t-1)\} \times 42$ VNs $\times 8$ connection layers
D	Computed intermediate VN-to-CN messages in column slice t 5-bit msg: $\{L_{vc}\} \times 42$ VNs $\times (1\text{-to-}4)$ processing layers
E	Outgoing connection layer messages from column t to $t+1$ 10-bit msg: $\{p_c(t), s_c(t), \min1_c(t), \min2_c(t)\} \times 42$ VNs $\times 8$ connection layers
F, G	Output hard decisions for VNs in current column slice t 1-bit decision: $\{\hat{C}_v\} \times 42$ VNs

CN+VN units in adjacent column slices is not any-to-any, but rather, is mandated by the column-to-column connectivity defined by the matrix. Hence, there is no multiplexer fan-out between connected CN+VN processors in adjacent columns along the same CN path. Depending on the code rate, only a subset of layers may be active. The rate 1/2 code requires all $8 \times 42 = 336$ CN paths, whereas the rate 13/16 code requires only $3 \times 42 = 126$ CN paths. Inactive layers/paths are disabled (turned off) through clock gating. Each column in the matrices shown in Fig. 5(a) has at most 4 active CN connections and, thus, each CN+VN processor requires at most 4 processing layers. Processing nodes in the last 4 columns require only 1, 2, or 3 processing layers due to the lower triangular matrix structure.

This architecture exploits the QC matrix structure by constraining routing to local interconnect between adjacent columns. Since each of the four matrices is derived from a single base matrix, multirate functionality is intrinsic to the hard-wired cyclic permutation networks between adjacent columns. The same wiring is used between successive columns for multiple code rates, thus eliminating additional permutation and control logic to switch between code rates. As the decoder switches from a low-rate code to a high-rate code, e.g., from rate 1/2 to rate 3/4, the unused wired layers of the high-rate code are disabled. The decoder continues to pass messages along the active processing layers, and hardware utilization in the combined processing nodes remains at either 100% (for rate 1/2, 5/8, and 3/4 codes) or 75% (for the rate 13/16 code) since the four rates have either 3 or 4 active processing layers.

D. Column Slice Architecture

Fig. 6 presents the architecture of a single column slice, which contains local memories and processing nodes for the VNs associated with that particular QC macrocolumn. Table II provides an overview of the messages exchanged between the CN+VN units and local memories in the column slice. Here, $p_c(t)$ and $s_c(t)$ are each 1 bit, and $\min1_c(t)$ and $\min2_c(t)$ are each 4 bits, such that each layer message is 10 bits. Input LLRs are buffered in through the Q_v memory write port. Output hard decisions are buffered out through the $\hat{C}_v$ memory read port. Outgoing update messages in column t are computed based on incoming messages from the previous column $t-1$ and values stored in the column-slice memories.

The proposed architecture implements a pipelined frame interleaving schedule with input/output (I/O) frame buffering

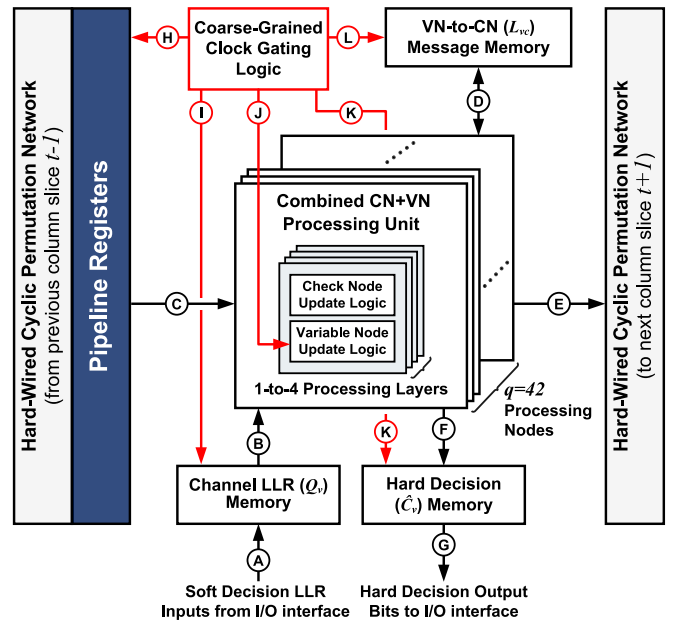


Fig. 6. Expanded detail of column slice t comprised of CN+VN processing units, local memory, and wired permutation networks between adjacent column slices. Pipeline registers are connected only to the first column in a column-slice pair. Hard-wired interconnect is specified by the parity-check matrix connectivity and does not contain multiplexing logic. The operations in column slice t are computed in one clock cycle. All column-slice memories are dual-ported register files with one read port and one write port.

TABLE III
MEMORY SPECIFICATION IN EACH COLUMN SLICE

Memory	Depth (Frames)	Data Bus Width (Bits)	Total Size (Kb)
Q_v Cols 0-15	16	42 VNs $\times$ 5 bits/message = 210	3.360
$\hat{C}_v$ Cols 0-15	16	42 VNs $\times$ 1 bit/decision = 42	0.672
L_{vc} Cols 0-11	8	42 VNs $\times$ 4 layers* $\times$ 5 b/msg = 840	6.720
L_{vc} Cols 12-13	8	42 VNs $\times$ 3 layers* $\times$ 5 b/msg = 630	5.040
L_{vc} Cols 14	8	42 VNs $\times$ 2 layers* $\times$ 5 b/msg = 420	3.360
L_{vc} Cols 15	8	42 VNs $\times$ 1 layers* $\times$ 5 b/msg = 210	1.680

* Active processing layers, as shown in Fig. 5(a).

to support continuous decoding without idle cycles. These features are described in Sections III-E and III-F. In each column slice, the input LLR Q_v and output hard decision $\hat{C}_v$ memories require a memory depth of 16 frames to support I/O frame buffering, while the VN-to-CN L_{vc} message memories require a memory depth of 8 frames to support the parallel decoding of 8 interleaved frames. The total memory requirements for input LLRs, output hard decisions, and VN-to-CN messages across all 16 column slices are 53.760, 10.752, and 95.760 Kb, respectively, for a total decoder memory of 160.272 Kb. Table III provides a parametric overview of each column-slice memory. These configurations are specific to IEEE 802.11ad, and will vary depending on the CN-VN connectivity defined by the matrix.

Coarse-grained clock gating logic in each column-slice pair disables the pipeline registers and local memories from switching when the current frame has successfully been decoded.

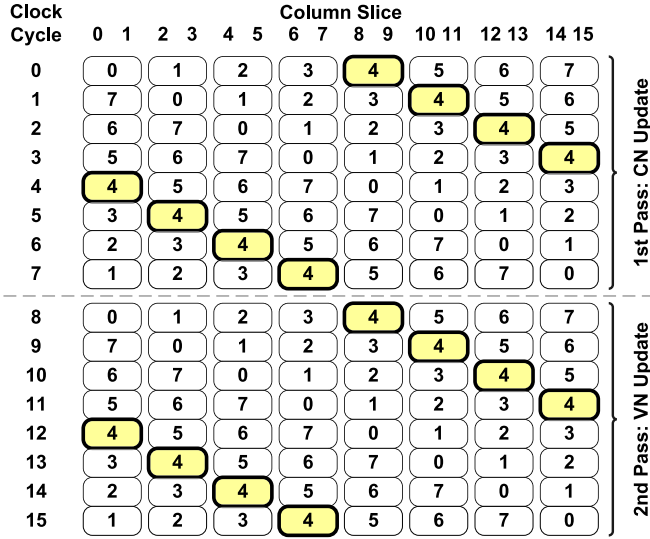


Fig. 7. Pipelined frame interleaving pattern through column-slice pairs over 16 clock cycles of one decoding iteration comprised of two passes. The number in each bubble indicates the frame index. Frame 4 highlights the cyclic frame-shifting property of the architecture.

Four independent clock gating cells are used in each column-slice pair to disable the input pipeline register bank to column t , and the Q_v , $\hat{C}_v$, and L_{vc} memories in columns t and $t + 1$, as highlighted by labels H, I, K, and L in Fig. 6, respectively. Label J in Fig. 6 shows the clock gating control for a pipeline register in the L_{vc} memory write path in the VN-update logic in each processing unit. When clock gating is active, the pair of column slices between two successive pipeline stages is effectively turned off. The clock gating pattern is identical for both columns t and $t + 1$ in a column-slice pair due to the pipelined frame interleaving described next.

E. Pipelined Frame Interleaving

The constant workload of the time-distributed decoding schedule enables message pipelining between successive column slices, since the number of columns is fixed for all code rates. As shown in Fig. 5(b), pipeline registers are placed after every two columns to satisfy the 1- μ s decoding latency requirement of the IEEE 802.11ad standard, assuming 10 decoding iterations. A single frame is shared by columns t and $t + 1$ in a column-slice pair. Since the computation in each column is independent and a constant number of hops is required to traverse each closed path, 8 independent frames can be interleaved without any memory access contention.

Fig. 7 presents the frame interleaving schedule where 8 interleaved frames are cyclically pipelined across 16 column slices. Both CN- and VN-update phases require 8 clock cycles to complete; thus, 16 cycles are required to complete a single iteration. Since each frame is independent, the frame-interleaved schedule ensures full hardware utilization without stall cycles. Frames of different code rates can also be interleaved simultaneously with minimal bypass routing configuration overhead.

For a fixed number of iterations, the minimum decoding throughput and maximum latency of a frame-interleaved

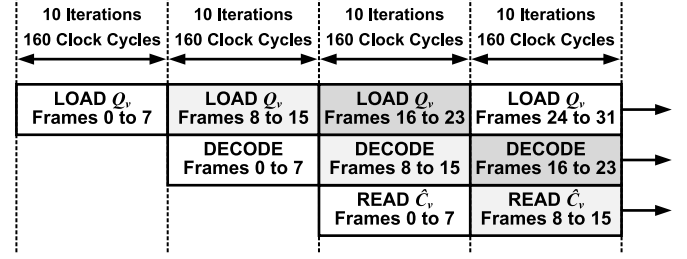


Fig. 8. Input/output frame buffering schedule, assuming a uniform decoding latency of 10 iterations with 16 clock cycles per iteration.

LDPC decoder are given by the following equations:

$$\text{Throughput} = \frac{\text{Frames} \times \text{Block Length} \times f_{\text{clk}}}{\text{Iterations} \times \text{Cycles Per Iteration}} \text{ (bits/s)} \quad (6)$$

$$\text{Latency} = \frac{\text{Iterations} \times \text{Cycles Per Iteration}}{f_{\text{clk}}} \text{ (seconds)}. \quad (7)$$

For a block length of 672 bits, the proposed decoder achieves a throughput of 6.78 Gb/s and a latency of 0.793 μ s with 8 interleaved frames, while operating at $f_{\text{clk}} = 202$ MHz clock rate and 10 iterations. This satisfies the maximum throughput and latency requirements of the IEEE 802.11ad standard.

F. Input/Output Frame Buffering for Continuous Decoding

The dual-ported channel LLR and hard decision memories enable I/O frame buffering such that the decoder runs continuously without any idle cycles. Fig. 8 presents a timing diagram of the pipelined I/O frame buffering schedule. The I/O latency is masked by the compute latency of the decoder. While the current 8 frames are being decoded, the next 8 frames are loaded in to the channel LLR Q_v memory, and the previous 8 frames are buffered out from the hard decision $\hat{C}_v$ memory. As shown in Table III, the Q_v and $\hat{C}_v$ memories have twice the depth compared to the VN-to-CN L_{vc} memory to accommodate this schedule.

G. Combined CN+VN Processing Unit Architecture

Fig. 9 presents the architecture of the combined CN+VN processing unit, where CN- and VN-update logic blocks share the memory and layer-message routing interfaces. The CN- and VN-update logic is partitioned independently. VN logic is disabled (turned off) during the CN phase, and vice versa. One clock cycle is required to perform either the CN or VN update in column slice t .

In every clock cycle, the combined CN+VN processing unit in column t first receives an incoming layer message $\{p_c(t-1), s_c(t-1), \min1_c(t-1), \min2_c(t-1)\}$ from its connected processing node in the previous column $t-1$. Either the CN- or VN-update is performed using the elements of the incoming layer message and the internal L_{vc} and Q_v values. The updated $\{p_c(t), s_c(t), \min1_c(t), \min2_c(t)\}$ layer message is then transmitted to its connected node in column $t+1$.

In the CN phase, stored L_{vc} values for each active processing layer are read in parallel from the L_{vc} memory. The magnitude of each L_{vc} value is compared with the $\min1_c(t-1)$ and $\min2_c(t-1)$ values of the incoming layer message, while the sign of each L_{vc} value is compared with the sign element

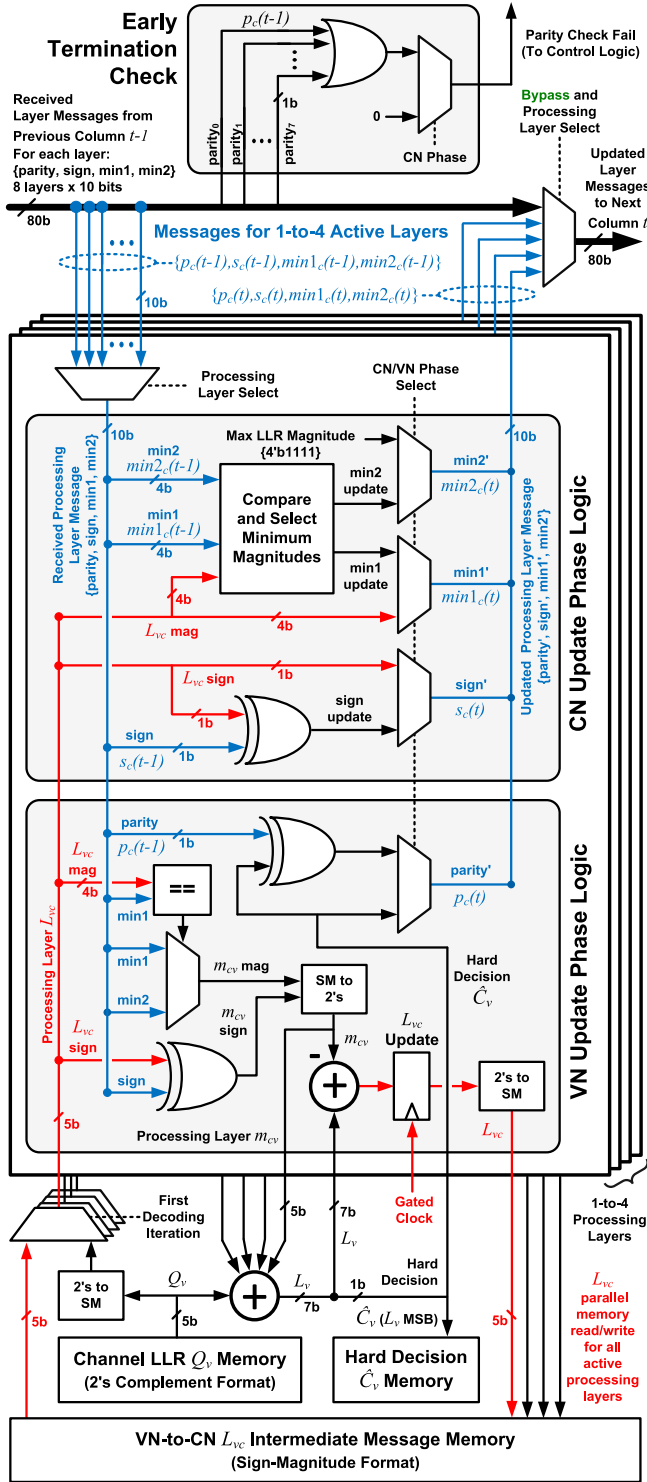


Fig. 9. Combined CN+VN processing unit for time-distributed piecewise decoding, showing CN- and VN-update phase logic, memory interfaces, and data permutation logic between processing units in successive column slices. A retiming register is included in the L_{vc} memory write path.

$s_c(t-1)$ of the incoming layer message. The updated $s_c(t)$, $min1_c(t)$, and $min2_c(t)$ values are transmitted to the next column slice. In the first decoding iteration, the L_{vc} values are not initialized and, thus, the Q_v LLR memory is read instead.

In the VN phase, the L_{vc} message for each active processing layer, hard decision $\hat{C}_v$ bit, and parity message element

$p_c(t)$ are updated based on the computed minimum sign and magnitude values in the previous CN phase. The parity $p_c(t)$ element of the layer message is updated, and transmitted to the next column, along with the unmodified $s_c(t-1)$, $min1_c(t-1)$, and $min2_c(t-1)$ elements that were received from the previous column. The early-termination parity check is performed in the first clock cycle of the CN-phase, starting from the second decoding iteration, once the $p_c(t)$ parity bits in each layer message have been computed and have returned home to their starting column $t = 0$ in the closed path traversal.

Both sign-magnitude (SM) and 2's complement number formats are used in the combined processing node logic. Each L_{vc} message is represented in a 5-bit SM format with 1 sign bit and 4 magnitude bits, to enable direct comparison of the sign and magnitude with the incoming $s_c(t-1)$, $min1_c(t-1)$, and $min2_c(t-1)$ values. Each Q_v LLR is represented in a 5-bit 2's complement format to avoid SM-to-2's complement conversion prior to the addition operation in the VN phase. The intermediate m_{cv} value in the VN phase is also represented by 5 bits in 2's complement format. The hard decision $\hat{C}_v$ is the most significant bit of the updated LLR L_{vc} .

The CN+VN processing unit exploits the spatial locality of L_{vc} and Q_v values stored in partitioned column-slice memories through a deterministic access pattern during the CN- and VN-update phases. The combined CN and VN logic minimizes routing complexity between processing units and eliminates additional data shifting logic. The time-distributed decoding schedule also eliminates the timing-constrained compare-select and XOR trees employed in traditional architectures to compute the minimum magnitudes, sign, and parity.

Fig. 10 presents the timing diagram for each frame j that is decoded through two CN+VN processing units in successive columns t and $t+1$ within a single pipeline stage over one clock cycle. The timing diagram highlights the individual operations in the CN- and VN-update phases, as well as the data dependency and message-passing sequence between processing units in successive columns t and $t+1$. All CN+VN units perform the same operation in each clock cycle.

H. Early Termination With Coarse-Grained Clock Gating

Early termination logic terminates the decoding of frames where the parity-check condition across all CNs is satisfied, i.e., $p_c = 0$ for every CN path as defined by (1). Early termination saves power at high SNR, where the majority of frames terminate within 1–3 iterations. Lower latency and higher throughput can be achieved if the decoder is configured to run continuously. In this case, the next 8 frames begin decoding immediately after the current 8 frames terminate.

The early termination check is performed independently at the starting column $t = 0$ for each interleaved frame. Clock gating turns off column slices, in which the current frame has terminated. Fig. 11 presents a sample frame termination pattern, which shows all eight frames terminating within 5 iterations. Frames that have terminated are not updated or cycled further.

The proposed frame-interleaved architecture addresses the challenges of designing a multi-Gb/s LDPC decoder with low-complexity interconnect and multirate reconfigurability, by introducing a time-distributed decoding schedule and combined CN+VN processing unit design. Section IV presents

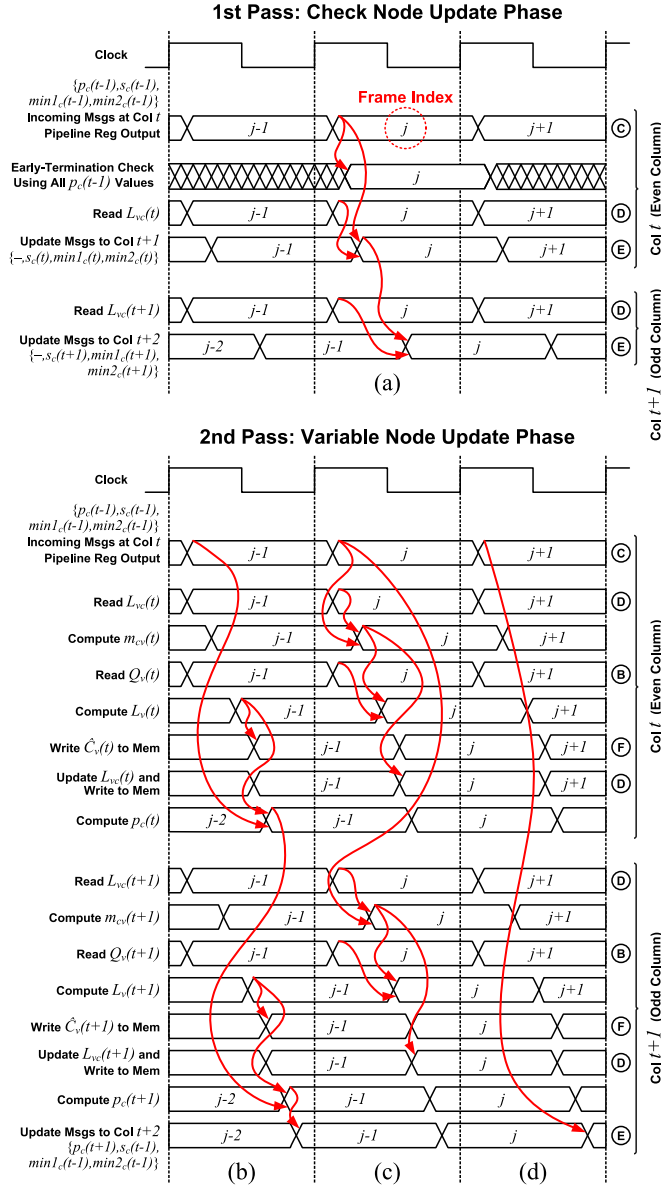


Fig. 10. Processing unit timing diagram for CN- and VN-update phases showing three independent frame updates over three clock cycles. Each CN+VN processing unit updates a single frame j in each clock cycle. All arrow-highlighted operations occur in each clock cycle and independently for each frame j , $j \in \{0, 1, \dots, 7\}$. Circled labels B–F correspond to the connections shown in the column slice architecture in Fig. 6. The following operations are highlighted. (a) Sign $s_c(t)$, first minimum magnitude $\min 1_c(t)$, and second minimum magnitude $\min 2_c(t)$ updates through column slice pair. (b) Parity $p_c(t)$ updates through column-slice pair. (c) Independent L_{vc} and $\hat{C}_v$ updates in columns t and $t+1$. (d) Propagation of sign, first minimum magnitude, and second minimum magnitude messages to the next column-slice pair without updates in columns t and $t+1$.

the physical implementation details and results of a proof-of-concept silicon test chip that demonstrates the low-power performance of the proposed architecture.

IV. PHYSICAL IMPLEMENTATION AND RESULTS

An LDPC decoder test chip was synthesized, placed-and-routed, and fabricated in a 28-nm CMOS technology as a proof-of-concept of the proposed architecture. The design contains 837 K gates and 160 Kb of embedded static

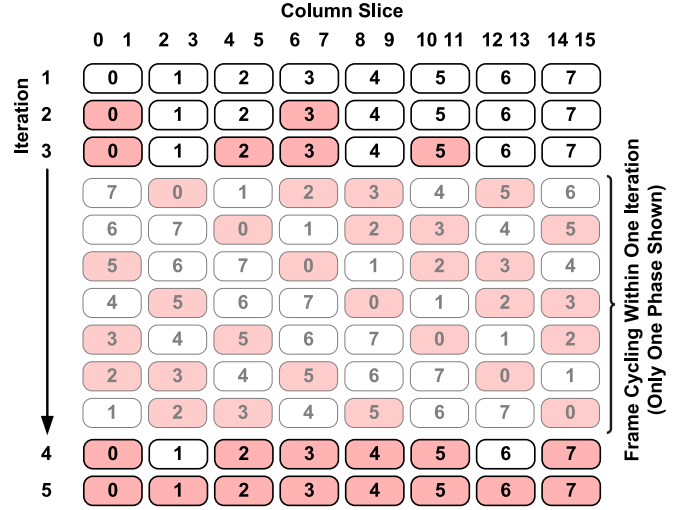


Fig. 11. Sample frame termination pattern in frame-interleaved architecture. The highlighted frames are assumed to have terminated. One iteration is performed over 16 clock cycles. Frames that have terminated are not updated in their current pipeline stage. Column slices in which the current frame has terminated are disabled through coarse-grained clock gating in each cycle.

random-access memory (eSRAM), which was generated using a commercial memory compiler. Embedded SRAMs were implemented instead of standard-cell flip-flops due to their higher bit-cell density, testability, and repairability to maximize post-fabrication yield. The decoder supports all four code rates and 24 throughput modes of the IEEE 802.11ad standard, while operating at a nominal 0.9-V supply and 202-MHz clock rate. Fig. 12 presents a die micrograph of the test chip. The decoder core in the fabricated test chip occupies an area of 3.41 mm², and contains two power domains to independently measure core logic and eSRAM power.

The core area of the fabricated test chip was significantly larger than necessary to accommodate the two decoupled power domains. The proposed design was also placed-and-routed without any timing violations across all PVT corners using a single power domain that allowed for higher placement density between standard cells and eSRAM blocks, achieving a core area of 1.99 mm². In comparison to the fabricated chip, the 1.99-mm² placed-and-routed design occupies 1.71× less area, has 0.82× total wire length, 0.85× standard cell area, and 0.52× buffer area. Given these area reductions and assuming uniform switching activity, the 1.99-mm² design would consume approximately 1.18× less power than the fabricated test chip. The results for both the actual measured power of the fabricated test chip and the estimated power of the reduced-area placed-and-routed design are presented here.

A. Error-Correction Decoding Performance

Fig. 13 presents the frame error rate (FER) and BER performance of the four codes. The input LLRs are quantized to 5 bits for both floating-point and fixed-point simulations.

B. Post-Silicon Power Measurements

The fabricated chip was tested on a Teradyne UltraFLEX-HD automated tester at a room temperature of 21 °C. The chip contains two test modes for at-speed functional verification and power measurement. In the functional test mode, channel

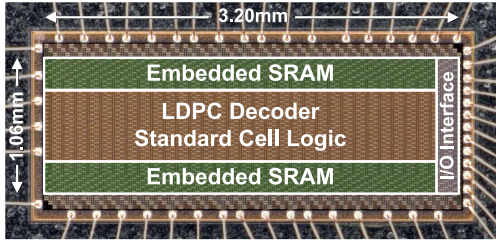


Fig. 12. Die micrograph of the fabricated test chip with wirebonds shown in exposed package. The decoder core (eSRAM, decoder logic, and I/O interface) occupies an area of 3.41 mm^2 ($3.20 \text{ mm} \times 1.06 \text{ mm}$). The total die size including pads is 4.78 mm^2 ($3.36 \text{ mm} \times 1.42 \text{ mm}$).

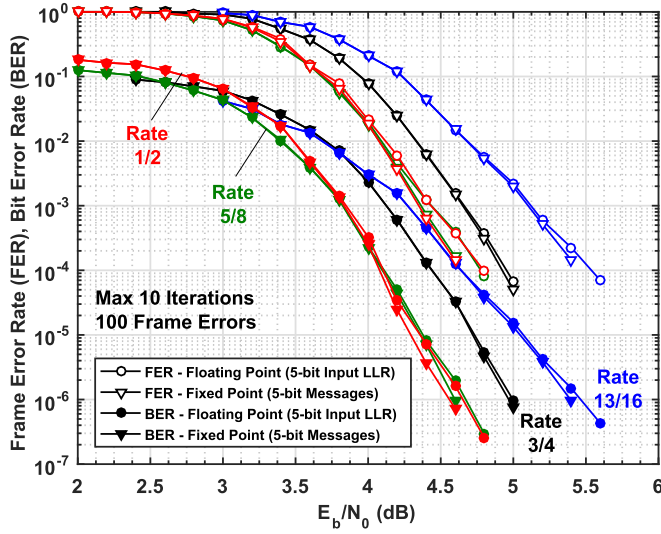


Fig. 13. FER and BER versus SNR under Min-Sum decoding for all four IEEE 802.11ad codes on BIAWGNC with maximum 10 decoding iterations. Channel SNR is normalized to energy per bit based on $E_b/N_0 = 1/(2\sigma^2 R_{\text{code}})$.

input LLRs are loaded through a shift-register I/O interface, the decoding is performed, and the output hard decision bits are shifted out. The captured hard decision bits are compared with a set of expected values predetermined through C++ simulation of a fixed-point LDPC decoder with a floating-point BIAWGNC model. The chip functionality was verified up to 300 MHz over 40 test cases: five SNR points in each of the four code rates, both with and without early termination. In the power test mode, decoded hard decision bits are not shifted out after each set of frames terminates. Instead, the decoder runs continuously such that once a set of frames terminates, the decoder immediately restarts the decoding cycle without any idle period. The supply current is sampled over 10 ms to obtain an accurate power measurement.

Fig. 14 presents the average power measured over ten typical-corner chips under nominal conditions. With early termination, overall power can be reduced by up to $1.43\times$ for the rate 1/2 code at $E_b/N_0 = 4.6 \text{ dB}$ and $2.93\times$ for the rate 13/16 code at $E_b/N_0 = 5.4 \text{ dB}$, while satisfying the maximum throughput specification of the IEEE 802.11ad standard. As the code rate increases, the power consumption decreases since the higher SNR enables more frequent early-termination clock gating. In addition, as shown in Fig. 5(a), the rate 13/16 code has only 3 active processing layers and, thus, less active logic than the other three codes.

TABLE IV

PERCENTAGE BREAKDOWN OF DESIGN AREA AND ESTIMATED POWER

Decoder Module	Area	Power with Early Termination Enabled *			
		Rate 1/2	Rate 5/8	Rate 3/4	Rate 13/16
Design Area and Measured Chip Power at BER = 10 ⁻⁶					
Core Area	1.99mm ²	279mW	176mW	112mW	104mW
Embedded SRAM Memories					
VN-to-CN L_{vc}	13.26%	17.36%	15.90%	15.01%	15.02%
Channel LLR Q_v	4.25%	1.95%	2.25%	2.13%	3.20%
Hard Decision $\hat{C}_v$	1.08%	0.58%	0.58%	0.54%	0.69%
Standard Cell Logic					
Column Slices	27.90%	11.53%	15.23%	15.78%	13.29%
Pipeline Registers	4.88%	7.30%	9.61%	9.71%	8.16%
Buffers/Inverters	11.21%	0.06%	0.08%	0.11%	0.04%
I/O and Control	0.43%	0.08%	0.10%	0.13%	0.06%
Core Filler Cells and Wired Routing					
Core Filler Cells	36.99%	N/A	N/A	N/A	N/A
Routing	N/A	59.79%	54.41%	54.69%	57.46%

* Power measured at $E_b/N_0 = 4.6 \text{ dB}$, 4.6 dB , 5.0 dB , and 5.4 dB for rates 1/2, 5/8, 3/4, and 13/16, respectively, at nominal 0.9 V supply and 202 MHz clock. Estimates are derived from power measurements and gate-level simulation reports.

TABLE V

DECODER PERFORMANCE AT $\text{BER} = 10^{-6}$ WITH EARLY TERMINATION

Code	Rate 1/2	Rate 5/8	Rate 3/4	Rate 13/16
E_b/N_0 (dB)	4.6	4.6	5.0	5.4
Avg. Iterations (Max 10) *	7	6	4	4
Nominal Conditions: $V_{DD_LOGIC}=0.9 \text{ V}$, $V_{DD_MEM}=0.9 \text{ V}$				
Clock Frequency (MHz)	202	202	202	202
Throughput (Gb/s) *	6.78	6.78	6.78	6.78
Max Latency (μs) *	0.793	0.793	0.793	0.793
Measured Power (mW)	279	176	112	104
Energy Efficiency (pJ/bit) *	41	26	16	15
Norm. Eff. (pJ/bit/iter) *	4.1	2.6	1.6	1.5
Low-Power Conditions: $V_{DD_LOGIC}=0.79 \text{ V}$, $V_{DD_MEM}=0.63 \text{ V}$				
Clock Frequency (MHz)	92	155	186	202
Throughput (Gb/s) *	3.09	5.20	6.25	6.78
Max Latency (μs) *	1.740	1.034	0.860	0.793
Measured Power (mW)	98	99	83	76
Energy Efficiency (pJ/bit) *	31	19	13	11
Norm. Eff. (pJ/bit/iter) *	3.1	1.9	1.3	1.1

* Throughput, latency, and efficiency calculations assume 10 decoding iterations, even though the decoder terminates and stops after the number of iterations specified in the "Avg. Iterations" row. The high-performance modes of the IEEE 802.11ad standard specify data rates of 3.08 Gb/s , 5.20 Gb/s , 6.24 Gb/s , and 6.76 Gb/s for the rate 1/2, 5/8, 3/4, and 13/16 codes, respectively.

Table IV presents an area breakdown by decoder module for the 1.99-mm^2 placed-and-routed design, and the estimated power consumed by each module in the 3.41-mm^2 fabricated test chip for four high-performance SNR points. Table V highlights the energy efficiency of both the nominal and low-power mode for four SNR points at a target BER of 10^{-6} . With clock-frequency and voltage scaling, the decoder satisfies the required data rates with up to $2.85\times$ less power than the nominal case. Through the multiframe I/O buffering technique shown in Fig. 8, the decoder can achieve higher throughput with lower latency by immediately starting to decode the next set of frames if the current set of frames terminates early. In this case, the power consumption is higher; however, the energy efficiency per bit remains the same, as shown in Table VI.

C. Comparison With the State of the Art

Table VI compares this work to the five most recently published CMOS-based, multi-Gb/s decoders for the IEEE

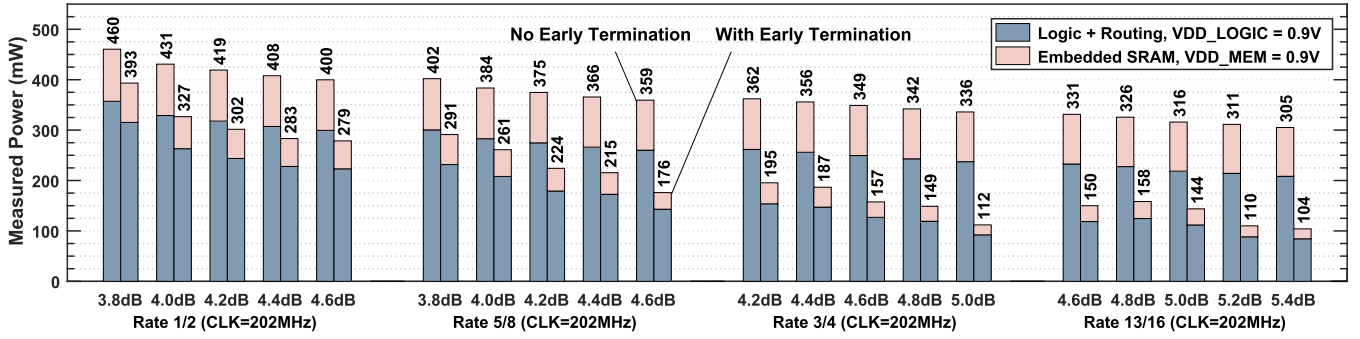


Fig. 14. Measured power of the 3.41-mm² fabricated test chip at nominal 0.9-V supply and 202-MHz clock rate, with and without early termination, delivering 6.78-Gb/s throughput. The five E_b/N_0 SNR points chosen for each code rate correspond to the five highest performance points in the waterfall region of each error-rate curve in Fig. 13. With early termination, eSRAM consumes between 18% and 23% of the total core power.

TABLE VI
COMPARISON OF LDPC DECODER IMPLEMENTATIONS FOR THE IEEE 802.11AD STANDARD

Specification	Weiner [32] ISSCC 2014	Park [33] JSSC 2014	Ajaz [37] APCCAS 2014	Li [38] ASSCC 2015	Motozuka [39] GlobalSIP 2015	This Work 2018			
Implementation	ASIC	ASIC	Place & Route	ASIP	ASIC	ASIC			
CMOS Technology Node	28nm FD-SOI	65nm	65nm	28nm	40nm LP	28nm			
Core Area (mm ²)	0.63	1.60	0.58	0.78	0.8	1.99 *			
Memory Type	Flip Flops	eDRAM	Flip Flops	N/A	Flip Flops	eSRAM			
Total Memory (Kbits)	N/A	33.6	7.875	N/A	12.096	160.272			
Pipeline Stages	5	5	1	5	3	8			
Interleaved Frames	2	2	1	1	1	8			
Supply Voltage (V)	1.1	0.94	1.1	0.9	1.1	0.9			
Clock Frequency (MHz)	260	360	400	470	220	202			
Block Length (Bits)	672	672	672	672	672	672			
Decoding Schedule	Flooding	Flooding	Layered	Layered	Flooding	Flooding			
Code Rate for Power Meas.	1/2	1/2	1/2	1/2	13/16	1/2 ^(a)		13/16 ^(b)	
Iterations	3.75	10	7	2	7	10 ^(c)	7 ^(d)	10 ^(e)	4 ^(f)
Throughput (Gb/s)	12.00	6.00	9.25	18.40	6.16	6.78	9.69	6.78	16.95
Latency (μ s)	0.112	0.224	0.073	0.037	0.109	0.793	0.555	0.793	0.317
Measured Power (mW)	180	373.6	272.9	166	203	279	399	104	260
Energy Efficiency (pJ/bit)	15.00	62.27	29.50	9.02	32.95	41.15		15.34	
Normalized Energy Efficiency (pJ/bit/iteration)	4.00	6.23	4.21	4.51	4.71	4.12	5.88	1.53	3.84
Estimated Norm. Energy Efficiency (pJ/bit/iteration)	4.00	6.23	4.21	4.51	4.71	3.49 *	4.98 *	1.30 *	3.25 *
Area Efficiency (Gb/s/mm ²)	19.05	3.75	16.09	23.59	7.70	3.41			
Max Reported Iterations	15	10	7	5	7	10			
Latency at Max Iters. (μ s)	0.448	0.224	0.073	0.0925	0.109	0.793			
Thpt. at Max Iters. (Gb/s)	3.00	6.00	9.25	7.36	6.16	6.78			
Decoding Algorithm	Offset Min-Sum	Offset Min-Sum	Min-Sum	Min-Sum	Min-Sum	Min-Sum			
Message Quantization (Bits)	5	5	L_{vc} : 4, m_{cv} : 2	N/A	5	5			
Multi Rate	Yes	No (Rate 1/2)	Yes	Yes	Yes	Yes			
BER for Rate 1/2 Code	10^{-6} at 4.4dB	10^{-6} at 3.6dB	10^{-5} at 3.6dB	10^{-6} at 4.0dB	10^{-6} at 4.0dB	10^{-6} at 4.6dB			
BER for Rate 5/8 Code	10^{-6} at 4.7dB	N/A	10^{-5} at 4.0dB	10^{-6} at 4.0dB	N/A	10^{-6} at 4.6dB			
BER for Rate 3/4 Code	10^{-6} at 4.5dB	N/A	10^{-5} at 4.3dB	10^{-6} at 5.0dB	N/A	10^{-6} at 4.6dB			
BER for Rate 13/16 Code	10^{-6} at 5.0dB	N/A	10^{-5} at 5.0dB	10^{-5} at 5.0dB	10^{-6} at 5.2dB	10^{-6} at 5.4dB			
Row/Column Parallelism	Row	Row	Row	Row	Column	Row and Column			
Permutation Network	Cyclic Shift Registers	Cyclic Shift Registers	Switch Network	Barrel Shift Network	Constrained Barrel Shifters	Cyclically Hard-Wired Partitions			
Parity-Check Matrix Modification	Row Re-Ordering	Row Merging	Column Re-Ordering	Row/Column Permutation	N/A	Bypass Routing			

* "Core Area" of "This Work" refers to the area of the placed-and-routed design using a single power domain. The fabricated test chip occupies an area of 3.41mm² and contains two on-chip power domains to independently measure logic and eSRAM power. Results reported for "This Work" in the row labeled "Estimated Norm. Energy Efficiency" correspond to the 1.18 $\times$ estimated reduction in power for the 1.99mm² design versus the actual measured chip power results presented in the "Measured Power," "Energy Efficiency," and "Normalized Energy Efficiency" rows.

(a) Power measured at $E_b/N_0 = 4.6$ dB and BER = 10^{-6} . (b) Power measured at $E_b/N_0 = 5.4$ dB and BER = 10^{-6} . (c) Decoder terminates early after 7 iterations, and remains idle for the remaining 3 iterations. (d) Decoder terminates early after 7 iterations, and immediately begins decoding next set of frames without any idle cycles. (e) Decoder terminates early after 4 iterations, and remains idle for the remaining 6 iterations. (f) Decoder terminates early after 4 iterations, and immediately begins decoding next set of frames without any idle cycles.

802.11ad standard [32], [33], [37]–[39]. All comparison works implement a partially-parallel architecture with a variant of the Min-Sum algorithm and achieve similar BER performance.

Several matrix modifications are applied among the comparison works, yielding different permutation network structures. This work reduces routing complexity by eliminating message

permutation logic using partitioned routing networks between adjacent column slices. While the 1.99-mm² placed-and-routed design occupies between 1.24× and 3.16× more unnormalized silicon area than the comparison works, the 3.41-mm² measured test chip achieves the lowest clock rate and highest normalized energy efficiency for the rate 13/16 code, and comparable normalized energy efficiency for the rate 1/2 code with acceptable latency. Despite the 160-Kb memory requirement, this work achieves low-power performance by turning off logic and memories via clock gating. As mentioned earlier, the placed-and-routed 1.99 mm² design with a single power domain offers an estimated 1.18× reduction in power versus the 3.41-mm² fabricated test chip. Based on this estimate, this work achieves the highest normalized energy efficiency compared with the five comparison works, as highlighted in the “Estimated Norm. Energy Efficiency” row in Table VI. Silicon area and power are not normalized to a particular CMOS technology node in Table VI, as Dennard scaling rules do not hold in newer nodes below 65 nm due to increased leakage current and adoption of dark silicon design techniques [14].

This section provides further comparison and discussion of the measured 3.41-mm² test chip versus the five comparison works. This work achieves approximately equal normalized energy efficiency for the rate 1/2 code compared with Weiner *et al.* [32], whose decoder was implemented using a fully-depleted silicon-on-insulator technology, which is known to provide superior power performance over conventional bulk CMOS [40]. Under worst-case channel conditions with a maximum of 15 decoding iterations, Weiner *et al.* achieve a throughput of only 3.00 Gb/s, which is below the maximum throughput specification for IEEE 802.11ad. This work achieves 1.5× higher normalized energy efficiency than the single-rate ASIC decoder by Park *et al.* [33], and similar normalized energy efficiency compared with the pre-silicon and application-specific instruction set processor decoder implementations for the rate 1/2 code by Ajaz and Lee [37] and Li *et al.* [38]. Their higher respective clock rates of 360, 400, and 470 MHz may introduce SoC integration challenges. The measured chip in this work achieves 3.1× higher normalized energy efficiency for the rate 13/16 code compared with the ASIC decoder by Motozuka *et al.* [39]. The LDPC decoder presented in this work achieves high energy efficiency, high error-rate performance, and the lowest clock rate in a modern bulk CMOS technology using commercial eSRAM.

V. CONCLUSION

This paper introduced a new partially parallel LDPC decoder architecture with pipelined frame interleaving and a path-unrolled message-passing schedule. The flooding Min-Sum algorithm was reformulated through a time-distributed computation over multiple processing units that contain both CN and VN update logic. Message permutation overhead and unstructured routing are minimized by exploiting the cyclic structure between adjacent macrocolumns in the QC matrix. The proof-of-concept CMOS-based decoder achieves a nominal throughput of 6.78 Gb/s with an energy efficiency between 1.53 and 4.12 pJ/bit/iteration at 0.9-V supply with a 202-MHz clock rate, which is ideally suited for SoC integration. By trading off interconnect complexity for high transistor

area, the proposed architecture introduces a new design strategy for LDPC decoders in modern CMOS technology nodes, where interconnect scaling has stagnated.

The architecture can be extended to support layered decoding at the expense of latency and control complexity. With a layered schedule, the VN update phase would still require only one pass; however, the CN update phase would require the same number of passes as the maximum VN degree.

The proposed architecture is also scalable to longer block-length codes, since 1) the critical timing path is not constrained by the expansion factor of the QC parity-check matrix; 2) the complexity of localized routing between successive columns is bounded by the number of connection layers; and 3) the high bit-cell density of eSRAM compensates for additional overhead in processing node logic. The architecture is reconfigurable and can be extended to support multiple standards with different parity-check matrices by including programmable shifters between successive columns. The overall memory size can be reduced by interleaving fewer frames in the architecture, e.g., four frames instead of eight frames. In this case, the decoder can maintain the same throughput by reducing the number of pipeline stages in each phase.

The architecture is not restricted to QC codes, but can also be applied more generally to random codes, or to codes that allow columnwise matrix partitioning as a way to enforce structure. Consider the example of a non-QC matrix with two CN connections to a VN in a single macrolayer, i.e., two “1” elements in a submatrix column. Here, the combined CN+VN processing unit simply needs additional CN-update logic to support the additional CN connected to the VN, while the VN-update logic remains the same. Additional wired routing is required to and from the additional CN-update logic; however, the overall path-unrolled message-passing schedule remains the same. This example can be extended to a random code, which can be partitioned into uniformly-sized macrocolumns.

Finally, the systolic nature of the proposed architecture is well suited for windowed decoding of spatially coupled LDPC codes [41]. Each decoding window of a spatially-coupled code can be mapped to an independent column-slice pair. Instead of decoding multiple frames, the decoder would slide the window as data moves from one pipeline stage to the next.

REFERENCES

- [1] R. G. Gallager, “Low-density parity-check codes,” *IRE Trans. Inf. Theory*, vol. 8, no. 1, pp. 21–28, Jan. 1962.
- [2] D. J. C. MacKay, “Good error-correcting codes based on very sparse matrices,” *IEEE Trans. Inf. Theory*, vol. 45, no. 2, pp. 399–431, Mar. 1999.
- [3] T. J. Richardson, M. A. Shokrollahi, and R. L. Urbanke, “Design of capacity-approaching irregular low-density parity-check codes,” *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 619–637, Feb. 2001.
- [4] S.-Y. Chung, G. D. Forney, Jr., T. J. Richardson, and R. Urbanke, “On the design of low-density parity-check codes within 0.0045 dB of the Shannon limit,” *IEEE Commun. Lett.*, vol. 5, no. 2, pp. 58–60, Feb. 2001.
- [5] *IEEE Standard for Information Technology—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*, IEEE Standard 802.11ac-2013, Dec. 2013, pp. 1–425.
- [6] T. Mohsenin, D. N. Truong, and B. M. Baas, “A low-complexity message-passing algorithm for reduced routing congestion in LDPC decoders,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 57, no. 5, pp. 1048–1061, May 2010.
- [7] K. Cushon, S. Hemati, C. Leroux, S. Mannor, and W. J. Gross, “High-throughput energy-efficient LDPC decoders using differential binary message passing,” *IEEE Trans. Signal Process.*, vol. 62, no. 3, pp. 619–631, Feb. 2014.

- [8] S. M. Rossnagel, R. Wisnieff, D. Edelstein, and T. S. Kuan, "Interconnect issues post 45 nm," in *IEDM Tech. Dig.*, Dec. 2005, pp. 89–91.
- [9] R. Brain, "Interconnect scaling: Challenges and opportunities," in *IEDM Tech. Dig.*, Dec. 2016, pp. 9.3.1–9.3.4.
- [10] K. Okada *et al.*, "Full four-channel 6.3-Gb/s 60-GHz CMOS transceiver with low-power analog and digital baseband circuitry," *IEEE J. Solid-State Circuits*, vol. 48, no. 1, pp. 46–65, Jan. 2013.
- [11] K. Zhang, X. Huang, and Z. Wang, "High-throughput layered decoder implementation for quasi-cyclic LDPC codes," *IEEE J. Sel. Areas Commun.*, vol. 27, no. 6, pp. 985–994, Aug. 2009.
- [12] Z. Chen, X. Peng, X. Zhao, L. Okamura, D. Zhou, and S. Goto, "A 6.72-Gb/s, 8pJ/bit/iteration WPAN LDPC decoder in 65 nm CMOS," in *Proc. 18th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2013, pp. 87–88.
- [13] *IEEE Standard for Information Technology—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*, IEEE Standard 802.11ad-2012, Dec. 2012, pp. 1–628.
- [14] M. B. Taylor, "A landscape of the new dark silicon design regime," *IEEE Micro*, vol. 33, no. 5, pp. 8–19, Sep./Oct. 2013.
- [15] M. Fossorier, M. Mihaljevic, and H. Imai, "Reduced complexity iterative decoding of low-density parity check codes based on belief propagation," *IEEE Trans. Commun.*, vol. 47, no. 5, pp. 673–680, May 1999.
- [16] R. M. Tanner, "A recursive approach to low complexity codes," *IEEE Trans. Inf. Theory*, vol. TIT-27, no. 5, pp. 533–547, Sep. 1981.
- [17] F. R. Kschischang, B. J. Frey, and H.-A. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 498–519, Feb. 2001.
- [18] D. Oh and K. Parhi, "Optimally quantized offset min-sum algorithm for flexible LDPC decoder," in *Proc. Asilomar Conf. Signals, Syst. Comput.*, Oct. 2008, pp. 1886–1891.
- [19] M. M. Mansour and N. R. Shanbhag, "High-throughput LDPC decoders," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 11, no. 6, pp. 976–996, Dec. 2003.
- [20] M. P. C. Fossorier, "Quasicyclic low-density parity-check codes from circulant permutation matrices," *IEEE Trans. Inf. Theory*, vol. 50, no. 8, pp. 1788–1793, Aug. 2004.
- [21] A. J. Blanksby and C. J. Howland, "A 690-mW 1-Gb/s 1024-b, rate-1/2 low-density parity-check code decoder," *IEEE J. Solid-State Circuits*, vol. 37, no. 3, pp. 404–412, Mar. 2002.
- [22] D. Oh and K. K. Parhi, "Low-complexity switch network for reconfigurable LDPC decoders," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 18, no. 1, pp. 85–94, Jan. 2010.
- [23] C. Roth, A. Cevrero, C. Studer, Y. Leblebici, and A. Burg, "Area, throughput, and energy-efficiency trade-offs in the VLSI implementation of LDPC decoders," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2011, pp. 1772–1775.
- [24] D. E. Hocevar, "A reduced complexity decoder architecture via layered decoding of LDPC codes," in *Proc. IEEE Workshop Signal Process. Syst.*, Oct. 2004, pp. 107–112.
- [25] T. Bhatt, V. Sundaramurthy, V. Stolpman, and D. McCain, "Pipelined block-serial decoder architecture for structured LDPC codes," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, vol. 4, pp. 225–228, May 2006.
- [26] A. Darabiha, A. C. Carusone, and F. R. Kschischang, "Power reduction techniques for LDPC decoders," *IEEE J. Solid-State Circuits*, vol. 43, no. 8, pp. 1835–1845, Aug. 2008.
- [27] Z. Wang and Z. Cui, "Low-complexity high-speed decoder design for quasi-cyclic LDPC codes," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 15, no. 1, pp. 104–114, Jan. 2007.
- [28] Y. Chen and K. K. Parhi, "Overlapped message passing for quasi-cyclic low-density parity check codes," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 51, no. 6, pp. 1106–1113, Jun. 2004.
- [29] L. Liu and C.-J. R. Shi, "Sliced message passing: High throughput overlapped decoding of high-rate low-density parity-check codes," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 55, no. 11, pp. 3697–3710, Dec. 2008.
- [30] M. Li *et al.*, "An area and energy efficient half-row-paralleled layer LDPC decoder for the 802.11AD standard," in *Proc. IEEE Workshop Signal Process. Syst.*, Oct. 2013, pp. 112–117.
- [31] S. Kumawat *et al.*, "High-throughput LDPC-decoder architecture using efficient comparison techniques & dynamic multi-frame processing schedule," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 62, no. 5, pp. 1421–1430, May 2015.
- [32] M. Weiner, M. Blagojevic, S. Skotnikov, A. Burg, P. Flatresse, and B. Nikolic, "27.7 A scalable 1.5-to-6Gb/s 6.2-to-38.1 mW LDPC decoder for 60 GHz wireless networks in 28 nm UTBB FDSOI," in *ISSCC Dig. Tech. Papers*, Feb. 2014, pp. 464–465.
- [33] Y. S. Park, D. Blaauw, D. Sylvester, and Z. Zhang, "Low-power high-throughput LDPC decoder using non-refresh embedded DRAM," *IEEE J. Solid-State Circuits*, vol. 49, no. 3, pp. 783–794, Mar. 2014.
- [34] P. Meinerzhagen, A. Bonetti, G. Karakostas, C. Roth, F. Giirkaynak, and A. Burg, "Refresh-free dynamic standard-cell based memories: Application to a QC-LDPC decoder," in *Proc. IEEE Int. Symp. Circuits Syst.*, May 2015, pp. 1426–1429.
- [35] D. Miyashita *et al.*, "An LDPC decoder with time-domain analog and digital mixed-signal processing," *IEEE J. Solid-State Circuits*, vol. 49, no. 1, pp. 73–83, Jan. 2014.
- [36] M. R. Li, C.-H. Yang, and Y.-L. Ueng, "A 5.28-Gb/s LDPC decoder with time-domain signal processing for IEEE 802.15.3c applications," *IEEE J. Solid-State Circuits*, vol. 52, no. 2, pp. 592–604, Feb. 2017.
- [37] S. Ajaz and H. Lee, "Multi-Gb/s multi-mode LDPC decoder architecture for IEEE 802.11ad standard," in *Proc. IEEE Asia Pacific Conf. Circuits Syst. (APCCAS)*, Nov. 2014, pp. 153–156.
- [38] M. Li *et al.*, "An energy efficient 18Gbps LDPC decoding processor for 802.11ad in 28 nm CMOS," in *Proc. IEEE Asian Solid-State Circuits Conf.*, Nov. 2015, pp. 1–5.
- [39] H. Motozuka, N. Yosoku, T. Sakamoto, T. Tsukizawa, N. Shirakata, and K. Takinami, "A 6.16Gb/s 4.7pJ/bit/iteration LDPC decoder for IEEE 802.11ad standard in 40 nm LP-CMOS," in *Proc. IEEE Global Conf. Signal Inf. Process. (GlobalSIP)*, Dec. 2015, pp. 1289–1292.
- [40] J. Colinge, "Fully-depleted SOI CMOS for analog applications," *IEEE Trans. Electron Devices*, vol. 45, no. 5, pp. 1010–1016, May 1998.
- [41] D. G. M. Mitchell, M. Lentmaier, and D. J. Costello, "Spatially coupled LDPC codes constructed from protographs," *IEEE Trans. Inf. Theory*, vol. 61, no. 9, pp. 4866–4889, Sep. 2015.



Mario Milicevic (S'11–M'17) received the B.A.Sc. and Ph.D. degrees in electrical engineering from the University of Toronto, Toronto, ON, Canada, in 2010 and 2017, respectively. His Ph.D. research focused on the design of error-correction decoders for integrated circuits and quantum cryptography.

He is currently a Staff Communication Systems Engineer at MaxLinear Inc., Irvine, CA, USA, where he is responsible for the development of algorithms and architectures for high-performance networking semiconductor products.



P. Glenn Gulak (S'82–M'83–SM'96) is currently a Professor with the Department of Electrical and Computer Engineering, University of Toronto, Toronto, ON, Canada. He has authored or coauthored over 150 publications in refereed journals and conference proceedings. His current research interests include the areas of algorithms, circuits, and CMOS implementations of high-performance baseband digital communication systems and hardware accelerators.

Dr. Gulak has received numerous teaching awards at the University of Toronto.